A5.3 Transaction ordering

A master can use the **AWID** and **ARID** transaction IDs to indicate its ordering requirements. The rules for the ordering of transactions are as follows:

- Transactions from different masters have no ordering restrictions. They can complete in any order.
- Transactions from the same master, but with different ID values, have no ordering restrictions. They can complete in any order.
- The data transfers for a sequence of read transactions with the same **ARID** value must be returned in the order in which the master issued the addresses, see *Read ordering*.
- The data transfers for a sequence of write transactions with the same **AWID** value must complete in the order in which the master issued the addresses, see *Normal write ordering* and *AXI3 write data interleaving* on page A5-79.
- There are no ordering restrictions between read and write transactions using a common value for **AWID** and **ARID**, see *Read and write interaction* on page A5-80.
- *Interconnect use of transaction identifiers* on page A5-80 describes how the AXI fabric extends the transaction ID values issued by AXI masters and slaves.

A5.3.1 Read ordering

At a master interface, read data from transactions with the same **ARID** value must arrive in the order in which the master issued the addresses. Data from read transactions with different **ARID** values can arrive in any order. Read data of transactions with different **ARID** values can be interleaved.

A slave must return read data for a sequence of transactions with the same **ARID** value in the order in which it received the addresses. In a sequence of read transactions with different **ARID** values, the slave can return the read data in any order, regardless of the order in which the transactions arrived.

The slave must ensure that the **RID** value of any returned data matches the **ARID** value of the address to which it is responding.

The interconnect must ensure that the read data from a sequence of transactions with the same **ARID** value targeting different slaves is received by the master in the order in which it issued the addresses.

The read data reordering depth is the number of addresses pending in the slave that can be reordered. A slave that processes all transactions in order has a read data reordering depth of one. The read data reordering depth is a static value that must be specified by the designer of the slave.

Note

There is no mechanism by which a master can determine the read data reordering depth of a slave.

A5.3.2 Normal write ordering

Unless a master knows that a slave supports write data interleaving, it must issue the data of write transactions in the same order in which it issues the transaction addresses. See *AXI3 write data interleaving* on page A5-79.

Note

- There is no mechanism by which a master can determine whether a slave supports write data interleaving. In AXI4, there is no support for write data interleaving.
- Most slave designs do not support write data interleaving and therefore must receive write data in the order in which they receive the addresses.

If the interconnect combines write transactions from different masters to one slave, it must ensure that it forwards the write data in address order.

Copyright © 2003, 2004, 2010, 2011 ARM. All rights reserved.
Non-Confidential

These restrictions apply even if the write transactions have different **AWID** values, and even if they come from different masters.

A5.3.3 AXI3 write data interleaving

Caution

AXI4 removes support for write data interleaving. In AXI4, all of the write data for a transaction must be provided in consecutive transfers on the write data channel. See *Removal of write interleaving support* on page A5-81.

With write data interleaving, a slave interface can accept interleaved write data with different **AWID** values. The write data interleaving depth is the number of addresses for which a slave can accept interleaved data.

When accessing a slave that supports write data interleaving, write data from different transactions that use the same **AWID** cannot be interleaved.

Note

As indicated in *Normal write ordering* on page A5-78, there is no mechanism by which a master, or any other AXI component, can determine whether a slave supports write data interleaving. Similarly, there is no mechanism by which the write interleaving depth of a slave can be determined.

For a slave that supports write data interleaving, the order in which it receives the first data item of each transaction must be the same as the order in which it receives the addresses for the transactions.

Note

If two write transactions with different **AWID** values access the same or overlapping addresses then the AXI3 specification does not define the processing order of those accesses. A higher-level protocol must ensure the correct order of transaction processing.

A master interface that generates write data using only one **AWID** value generates all write data in the order in which it issues the write addresses. However, a master interface can interleave write data with different **WID** values if the slave interface has a write data interleaving depth greater than one.

To avoid possible deadlock, a slave interface that supports write data interleaving must continuously accept interleaved write data. It must never stall the acceptance of write data in an attempt to change the order of the write data.

Usage models for write data interleaving

Write data interleaving can prevent stalling when the interconnect combines multiple streams of write data targeting the same slave. For example, the interconnect might combine a write data stream from a slow source with another write data stream from a fast source. By interleaving the two write data streams, the interconnect can improve system performance.

For most masters that can control the generation of the write data, write data interleaving is not necessary. Such a master can generate the write data in the order in which it generates the addresses. However, a master interface that is transferring write data from different sources that have different speeds might interleave the sources to make maximum use of the interconnect.

Copyright © 2003, 2004, 2010, 2011 ARM. All rights reserved.
Non-Confidential

A5.3.4 Read and write interaction

AXI has no ordering restrictions between read and write transactions. They can complete in any order, even if the **ARID** value of a read transaction is the same as the **AWID** value of a write transaction.

If a master requires a given relationship between a read transaction and a write transaction then it must ensure that the earlier transaction is complete before it issues the later transaction. A master can only consider the earlier transaction is complete when:

- for a read transaction, it receives the last of the read data
- for a write transaction, it receives the write response.

Sending all of the write data for the transaction must not be taken as indicating completion of that transaction.

Note

Typically, when writing to a peripheral, a master must wait for earlier transactions to complete before switching between read and write transactions that must be ordered.

For reads and writes to memory, a master might implement an address check against outstanding transactions, to determine whether a new transaction could be to the same, or overlapping memory address. If the read and write transactions do not overlap, then the master can start the new transaction without waiting for the earlier transactions to complete.

A5.3.5 Interconnect use of transaction identifiers

When a master is connected to an interconnect, the interconnect appends additional bits to the **ARID**, **AWID** and **WID** identifiers that are unique to that master port. This has two effects:

- masters do not have to know what ID values are used by other masters, because the interconnect makes the ID values used by each master unique, by appending the master number to the original identifier
- the ID identifier at a slave interface is wider than the ID identifier at a master interface.

For read data, the interconnect uses the additional bits of the **RID** identifier to determine which master port the read data is destined for. The interconnect removes these bits of the **RID** identifier before passing the **RID** value to the correct master port.

For write response, the interconnect uses the additional bits of the **BID** identifier to determine which master port the write response is destined for. The interconnect removes these bits of the **BID** identifier before passing the **BID** value to the correct master port.

A5.3.6 Width of transaction ID fields

The width of transaction ID fields is IMPLEMENTATION DEFINED. However, this specification recommends the following transaction ID field widths:

- for master components, implement a transaction ID field up to four bits
- for master port numbers in the interconnect, implement up to four additional bits of transaction ID field
- for slave components, implement eight bits of transaction ID field support.

For masters that support only a single ordered interface, it is acceptable to tie the transaction ID field outputs to a constant value, for example, tie to zero.

For slaves that do not make use of the ordering information and process all transactions in order, the transaction ID functionality can be added without changing the base functionality of the slave.

Copyright © 2003, 2004, 2010, 2011 ARM. All rights reserved.
Non-Confidential